idea-harness

UI / UX Creative Direction

Version: 1.1
Direction: Calm Command Center
Product principle: Calm on the surface. Rigorous underneath.

---

1. Design Thesis

idea-harness should not look or behave like:

- Jira with AI
- Trello with agents
- a chat app
- an IDE full of autonomous-agent terminals
- a generic observability dashboard

The user is not managing agents.

The user is developing an idea.

The interface should answer five questions quickly:

```
What are we building?

What have we learned?

What changed?

What still needs judgment?

What happens next?
```

Everything else is progressively disclosed.

---

2. Emotional Goal

Using idea-harness should feel like supervising a very strong product and engineering organization that works independently but keeps the user in control.

Desired feelings:

```
clarity
momentum
confidence
control
focus
```

Avoid:

```
noise
agent theater
constant alerts
approval fatigue
visualized complexity for its own sake
```

---

3. Visual Direction

Reference qualities:

```
Linear
  precision and restraint

GitHub
  information structure and review mechanics

Raycast
  speed and keyboard-native interaction

Stripe documentation
  typography and technical clarity
```

Runtime/operator screens may become denser, but the product-development experience should have more breathing room.

Avoid:

- AI gradients
- glowing orbs
- robots
- sparkles

- anthropomorphic agent avatars
- cyberpunk styling
- giant status dashboards

---

4. Visual Language

Use:

- neutral surfaces
- restrained borders
- high-quality typography
- subtle depth
- limited accent color
- strong spacing hierarchy
- compact metadata
- large readable product prose

State color is functional.

Suggested semantic use:

```
Active        blue/accent
Passed        green
Attention     amber
Blocked       red
Neutral       grayscale
```

Do not use color as the only state indicator.

---

5. Typography

Human/product information:

- larger type
- comfortable line height
- document-quality typography

Examples:

```
problem statement
decision
```

```
requirement
Current Understanding
PRD
design rationale
```

Machine/operator information:

- smaller
- compact
- optionally monospace

Examples:

```
REQ-17
RUN-81
CS-14
paths
hashes
model
tokens
executor
```

This creates a visual distinction between:

```
what humans reason about
```

and:

```
how the machinery operates
```

---

6. Navigation

Primary:

```
Ideas
Decisions
Activity
```

Secondary/operator:

```
Runtime
Settings
```

Avoid making:

```
Reviews
Agents
Artifacts
Graph
Tasks
```

all global top-level navigation.

Those are contextual views within the product.

---

7. Ideas

`Ideas` is the portfolio entry point.

When many initiatives exist, use a board:

```
Explore
Define
Design
Build
Verify
Done
```

The board is not the primary screen for developing a single idea.

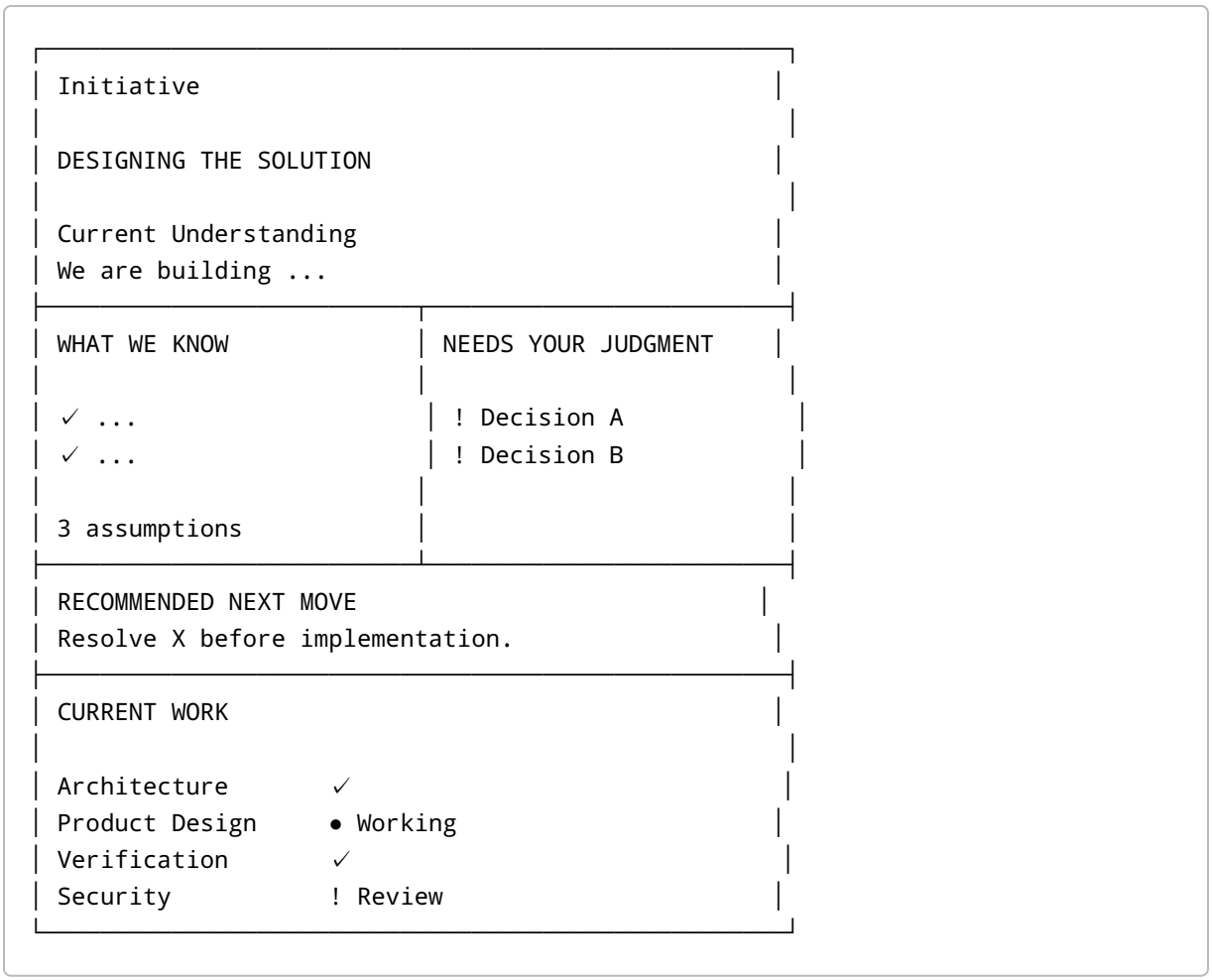
---

8. Initiative Home

Opening an initiative should land on its **Initiative Home**.

This is the primary product-development interface.

Recommended layout:



---

9. Current Understanding

This should be a signature component.

It is the continuously updated representation of:

- what is being built
- target user
- value
- major constraints
- key decisions
- unresolved questions

Users returning after hours or days should orient themselves here.

---

10. Recommended Next Move

Visible when a consequential next action exists.

Example:

```
RECOMMENDED NEXT MOVE

Resolve how conflicting decisions are handled.

Why
Storage, UX, and verification depend on it.

[Review decision]
```

This translates workflow complexity into product language.

---

11. Decisions

The global Decisions view is for human judgment.

Categories:

```
Need Your Decision
Blocked
Recommended Changes
For Awareness
```

Do not call every item an approval.

A product decision and permission request are different concepts.

---

12. Decision Card

Example:

```
SHOULD SHARING LINKS EXPIRE?

Product
```

```
Optional expiration.

Security
Default expiration.

UX
No expiration unless selected.

Recommendation
Default to 30 days, with "Never" available.

Why
Reduces accidental long-lived access without making
sharing cumbersome.

Affected
Security
Product Design
API

[Accept recommendation]
[Choose alternative]
[Discuss]
```

The system coordinates expert perspectives before presenting them.

---

13. Agent Conversations

Direct conversations with individual experts may exist as an advanced capability.

They should not be the default coordination model.

The user should not have to manually arbitrate:

```
PM chat
Architect chat
Designer chat
Security chat
```

idea-harness should synthesize those perspectives first.

---

14. Board

For portfolio orientation.

Cards should be compact and operational.

Example:

```
┌──────────────────────────────────┐
│ Usage-based billing              │
│                                  │
│ DESIGN                           │
│                                  │
│ Architecture        ✓            │
│ Product Design      •            │
│ Verification        ✓            │
│ Security            !            │
│                                  │
│ 2 unresolved decisions           │
└──────────────────────────────────┘
```

Do not emphasize models or agents.

---

15. No Fake Percentages

Avoid:

```
72% complete
```

unless tied to deterministic work.

Prefer:

```
Explore  ✓
Define   ✓
Design   •
Build    ○
Verify   ○
```

```
2 unresolved decisions
1 open high-risk issue
```

Uncertainty is often more meaningful than percentage completion.

---

16. Uncertainty Surface

Initiative Home should make uncertainty visible.

Compact indicators:

```
Open questions       3
Assumptions          4
Unresolved risks     1
Decisions needed     2
```

This communicates convergence better than task count alone.

---

17. What Changed

Another signature interaction.

Default Activity view should prioritize semantic change.

Example:

```
SINCE YOU LAST LOOKED

PRODUCT
• REQ-08 became mandatory.
• Mobile support moved out of scope.

DESIGN
• Passkeys replaced sessions.

BUILD
• Three changes accepted.
• One rejected after security review.
```

```
QUALITY
• New integration test covers REQ-08.
```

Raw event timeline remains available.

---

18. Activity

Activity combines:

Semantic summary

What materially changed.

Timeline

Chronological events.

Example:

```
10:03 Research started
10:07 Research completed
10:08 Review found a high-severity issue
10:12 Research v2 approved
10:15 PRD started
```

Timeline should ship before sophisticated graph visualization.

---

19. Workbench

For deeper initiative work.

Suggested contextual sections:

```
Overview
Requirements
Decisions
Artifacts
Changes
```

```
Verification
Timeline
```

Do not default to exposing all internal technical entities.

---

20. Artifact Experience

Artifacts should be beautiful readable documents with structured content visibly embedded.

Example PRD:

```
Problem
_______

...

Users
_____

...

Requirements                          12
──────────────────────────────────────────

REQ-01  Preserve source provenance    MUST
REQ-02  Allow human correction        MUST
REQ-03  Support Slack                 SHOULD

Assumptions                           3
──────────────────────────────────────────

ASM-01  Users want automation         MEDIUM

Open Questions                        2
──────────────────────────────────────────

...
```

This communicates that idea-harness understands the product rather than merely generating markdown.

---

21. Contextual Inspector

Right rail should change according to the object.

PRD

```
Review Findings
Requirements
Assumptions
Decisions
```

Technical Design

```
ADRs
Risks
Requirement Coverage
Security
```

ChangeSet

```
Tests
Requirements Addressed
Review Findings
Scope Deviations
```

Avoid a universal inspector with a dozen sections.

---

22. Semantic Diff

Artifact comparison should focus on meaning.

```
REQUIREMENT ADDED
REQ-14 ...

REQUIREMENT CHANGED
REQ-08 ...

DECISION SUPERSEDED
ADR-4 → ADR-9
```

Raw file diff is secondary.

---

23. Review UX

Reviews should feel closer to excellent code review than AI chat.

Finding:

```
HIGH · Authorization
HIGH CONFIDENCE

Users can revoke sharing links they do not own.

Evidence
share_handler.go:81

Requirement
REQ-14

Impact
Cross-project authorization violation.

Suggested resolution
Verify project ownership before revocation.

[Fix]
[Not applicable]
[Accept risk]
[Discuss]
```

Resolved:

```
RESOLVED

CS-84 addresses this finding.

[View change]
```

---

24. Risk Presentation

No numeric pseudo-precision.

Prefer:

```
HIGH RISK

Authorization
Public API
```

Click/hover explains why.

---

25. Why

`Why?` should be pervasive.

Examples:

```
Why is this requirement here?
Why is Security involved?
Why is this stale?
Why can't we build yet?
Why was this model selected?
Why did this ChangeSet fail?
```

Use a side panel for quick causal explanations.

---

26. Explain / Why / Impact / Trace

UI concept:

```
Explain   What is this?
Why       Why did this happen?
Impact    What depends on this?
Trace     Where did it come from?
```

These should feel consistent across all object types.

---

27. Product Graph

The graph powers the interface but should mostly stay invisible.

Use it for:

- Why
- Impact
- Trace
- staleness
- semantic diff
- requirement coverage

Offer raw visualization as an expert drill-down.

Avoid making node graphs the primary product metaphor.

---

28. Runtime

Runtime is the dense operator interface.

Example:

```
ARCHITECTURE

Staff Architect
Claude Opus

Reviewing TechnicalDesign v3

Current
Checking rollback strategy

Recent
✓ Evaluated schema
✓ Reviewed API compatibility
● Checking failure recovery
```

Do not show model internal reasoning.

---

29. Agent Identity

Role first.

Model second.

Good:

```
Staff Architect
Claude Opus
```

Avoid:

```
Archie 🤖
```

The product is about expertise, not simulated personalities.

---

30. Models

Model names belong mainly in:

- Runtime
- Inspect
- Costs
- Debugging
- Settings

The primary idea-development flow should not look like a model router.

---

31. Progress Feedback

Avoid generic spinners.

Prefer operational progress:

```
Architecture review

✓ Schema
✓ API compatibility
● Rollback strategy
○ Operational failure modes
```

Do not expose hidden chain-of-thought.

---

32. Empty State

Instead of:

```
No initiatives
```

show:

> **What are you thinking about building?**

Input:

```
Describe the idea, feature, problem, or change...
```

Starting contexts:

```
Idea
Existing folder
Requirement
Bug
```

---

33. Greenfield Setup

Example:

```
NEW WORKSPACE

No source code yet.
No Git repository.

We'll start by understanding the idea.

[Start exploring]
```

---

34. Existing Project Setup

Example:

```
EXISTING PROJECT

TypeScript
React
Fastify
PostgreSQL
Vitest
Git

✓ Test commands detected
✓ Architecture analyzed
! No engineering standards configured

[Use recommended defaults]
[Review setup]
```

---

35. Autonomy Controls

Settings should describe outcomes, not vague labels alone.

Example:

```
SUPERVISED

You approve:
• execution plans
• high-risk source changes

AI handles:
• low/medium-risk reviews
• routine remediation
• specialist consultation

Security invariants are always enforced.
```

---

36. Command Palette

Keyboard interaction should be first-class.

```
⌘K
```

Actions:

```
New idea
Open decisions
Ask why
Show what changed
Pause execution
Inspect current run
Open requirement
Switch project
```

---

37. CLI → UI Deep Links

Potential commands:

```
idea open REQ-17
idea open CS-18
idea open DEC-7
```

This reinforces a seamless developer experience.

---

38. Notifications

Default pull-first.

Use:

- Decision badge
- Blocked state
- completion summary

Optional desktop notifications for:

- human judgment required
- initiative completed
- critical review failure

Avoid noisy agent-progress notifications.

---

39. Accessibility

The idea-harness UI itself must support:

- keyboard navigation
- visible focus
- semantic labels
- screen readers
- contrast
- non-color state communication
- appropriate target sizing
- responsive layouts
- reduced motion preferences

---

40. Responsive Strategy

Primary design target:

```
desktop
```

because of document and review density.

Tablet should remain functional.

Mobile may support:

```
Decisions
Activity
Approvals
Status
```

later, but full authoring is not a v0.1 requirement.

41. Visual Density

Three density levels effectively emerge:

Initiative Home

Calm, spacious.

Workbench / Review

Moderate density.

Runtime

High density.

Do not use Runtime density everywhere.

---

42. Brand Personality

idea-harness should communicate:

```
judgment
discipline
precision
momentum
clarity
```

Not:

```
magic
autonomy theater
futurism
playfulness
```

The visual brand should feel credible beside serious developer infrastructure products.

---

43. Design Principle Checklist

Every new feature should ask:

Is this something the user needs to know now?

If not, hide it one level deeper.

Is this a product decision or system metadata?

Product decision gets visual priority.

Can idea-harness synthesize this instead of making the user coordinate agents?

If yes, synthesize it.

Does this communicate uncertainty honestly?

Avoid fake precision.

Does this expose evidence?

Important claims and reviews should be traceable.

Can the user recover?

Failures should show next actions.

---

44. Signature Interactions

The product should become recognizable through:

```
Current Understanding
Recommended Next Move
Decisions
What Changed
Why
Semantic Diff
Adversarial Review
```

These are more important to the identity than the Kanban itself.

---

45. Final UX Philosophy

The interface should embody:

> **Show me the idea, what we've learned, what changed, what remains uncertain, what needs my judgment, and what happens next. Hide the machinery until I ask for it.**

The user should never feel like the project manager for a collection of AI agents.

They should feel like the product leader of an unusually capable team.